TP7 — Administración y Seguridad en PostgreSQL (Unidad 5)

Institución: UTN – Facultad Regional Mendoza (FRM) | **Cátedra:** Base de Datos II | **Ciclo:** 2026
Docente: Sergio Neira | **Alumno:** Máximo Juan Cruz Quiroga (maximoj.quiroga@alumnos.frm.utn.edu.ar)

1. Parte A — Especificación e Implementación del Modelo de Roles (Mínimo Privilegio)

Se diseñó e implementó en **sql/roles.sql** un esquema de control de acceso basado en roles (RBAC) sobre el modelo canónico de Food Store (*categorias, clientes, productos, pedidos, detalles_pedido, usuarios*), revocando todos los permisos por defecto de *PUBLIC* y aplicando el Principio de Mínimo Privilegio (PoLP):

Rol	Tipo	Privilegios Efectivos y Restricciones
rol_app_lectura	NOLOGIN	SELECT sobre las 5 tablas de negocio y 3 vistas operativas; sin acceso a usuarios.
rol_app_escritura	NOLOGIN	INSERT, UPDATE en tablas de negocio (sin DELETE por regla R7); USAGE en secuencias; EXECUTE fn_autenticar.
rol_soporte	NOLOGIN	SELECT por columna en usuarios (excluye contrasena), UPDATE(activo) y EXECUTE fn_resetear_password.
rol_reportes	NOLOGIN	SELECT únicamente sobre v_reporte_ventas_cliente, v_usuario_publico y mv_resumen_ventas_categoria_mes.
rol_auditoria	NOLOGIN	Membresía en pg_monitor; sin acceso a tablas de negocio ni a pg_authid.rolpassword.
app_web	LOGIN (20)	Hereda rol_app_lectura y rol_app_escritura; sin privilegios DDL ni administración.
admin_datos	LOGIN (5)	ALL sobre objetos de public y CREATE en el esquema; sin atributo SUPERUSER.

2. Parte B — Configuración y Análisis de Auditoría Nativa

Mediante **ALTER SYSTEM SET** y **pg_reload_conf()** se activaron en caliente *log_connections = on*, *log_disconnections = on*, *log_statement = all* y *log_line_prefix = '%m [%p] %u@%d %r %a '*. La secuencia **sql/auditoria_test.sql** permitió verificar en **docs/log_auditoria_prueba.utf8.txt** la traza completa (quién, qué, cuándo y desde dónde). Se documentó que los argumentos literales de funciones quedan en texto plano en el log, por lo que el archivo debe resguardarse con permisos estrictos de sistema operativo (0600/ACL NTFS) y restringirse en el motor a superusuarios.

3. Parte C — Simulacro de Incidente, Reconstrucción con IA y Contención

El guion **sql/simulacro_incidente.sql** ejecutó 3 intentos fallidos contra *fn_autenticar* (retornando NULL), 1 login exitoso, lectura masiva bajo *rol_soporte* sobre *public.usuarios* y un intento de escalamiento (*SET LOCAL ROLE app_web; GRANT admin_datos TO app_web;*) que fue bloqueado por PostgreSQL con **SQLSTATE 42501**. El log anonimizado (**docs/log_simulacro_anonimizado.utf8.txt**) se analizó con IA: el asistente reconstruyó con precisión la secuencia cronológica, pero su sugerencia de bloquear automáticamente la IP de origen fue desestimada por criterio humano ante el riesgo de denegación de servicio (DoS) tras NAT/proxies, priorizando el aislamiento de sesión y la rotación de la credencial afectada vía *fn_resetear_password*.

4. Parte D — Protección de Datos Personales, Anonimización y Equivalencia

Los scripts **sql/usuario_anon.sql** y **sql/usuario_anon_cobertura.sql** transforman *public.usuarios* en una estructura anonimizada determinista que omite *contrasena*, *telefono*, nombres y correos reales, conservando únicamente *rol*, *activo* y *mes_alta*. La equivalencia analítica entre las consultas sobre la tabla anonimizada y la tabla real se demostró matemáticamente con **EXCEPT ALL** bidireccional (0 discrepancias tanto en la muestra base de 2 filas como en el ensayo ampliado de 8 filas y 5 grupos rol × mes).

5. Parte E — Bitácora de IA y Validación Crítica

En **BITACORA.md** se consolidaron las interacciones en la tabla reglamentaria de 3 columnas (Prompt, Respuesta y Validación Humana), destacando el rechazo de propuestas inseguras como *GRANT ALL*

PRIVILEGES a roles de aplicación y *GRANT SELECT ON usuarios TO rol_soporte*.